

COMS BC1016

Introduction to Computational Thinking and Data Science

Lecture 8: Joining Tables and Conditionals

Sep 30, 2025

Copyright © 2026 Barnard College

October 5, 2026

Logistics

- Checking in:
 - Any questions/comments on homeworks or labs?
 - HW 2 is due on Wednesday, Lab 3 was due last week
 - Does the amount of office hours feel sufficient?
- Please contact me or your class dean ASAP if there are any upcoming religious holidays I should be aware of
- Please contact CARDS/ODS for disability accommodations

Midterm Info

- Midterm is in 3 weeks on **Monday, October 26**
 - Exam will cover material up until **Wednesday, October 14**
 - TAs will lead a **midterm review** during class **Wednesday, October 21**
- In-class paper exam
 - You can bring a single 5"x8" index card with notes (to be submitted along with your exam)
 - Front and back, can be typed or handwritten
 - I will be bringing 5"x8" index cards beginning next class for anyone who wants one

Midterm Info

- You will *not* be expected to write complicated programs from scratch
- You may be asked to **identify issues** in snippets of code and be asked **how to fix them**
- You are expected to be familiar with **concepts/syntax that we've dedicated class time to** (e.g., functions, converting data types, basic operations, arrays)

Midterm Info

- You are not expected to memorize the data8 Python reference sheet.
I will *not* try to trip you up on small details like `.column` vs `.select`
- You are expected to be familiar with **anything in the slides** (e.g., groups, pivots, histograms, binning, creating tables, creating arrays, ...)
- You should be able to **draw/describe what output** is being produced (possibly given a copy of the relevant lines from the reference sheet)
 - You should generally know how a Table is formatted, what different visualizations look like, ...

Chart Selection Exercise

We have NYC weather data from 2019 as shown below (from [Kaggle](#))

Which type of chart (line, scatter, bar, histogram) would best help you answer to each question?

- Do days with hotter highs also tend to have hotter lows?
- How do the number of rainy days compare with the number of snowy days?
- What percent of days have a high of at least 75 degrees?

date	tmax	tmin	tavg	condition
1/1/19	60	40	50	rainy
2/1/19	41	35	38	
3/1/19	45	39	42	
4/1/19	47	37	42	
5/1/19	47	42	44.5	rainy
6/1/19	49	32	40.5	
7/1/19	35	26	30.5	
8/1/19	47	35	41	rainy
9/1/19	46	35	40.5	rainy
10/1/19	35	30	32.5	

Joining Two Tables

Joining Two Tables

Sometimes data about the same individuals are in different tables

- `join` combines the two datasets together
- Entries that do not appear in both tables are not included in the new table

To combine entries from `table1` and `table2` based on columns `c1` and `c2`

– `table1.join(c1, table2, c2)`

join Example

bubble_t teas

cafe	drinks	prices
Gong Cha	Matcha Tea Latte	5.75
Tea Magic	Oolong Milk Tea	8
Hey Tea	Coconut Mango Boom	6.49
Moge Tee	Taro Milk Tea	7.45

discounts

% off	location
10	Gong Cha
25	Hey Tea
5	Moge Tee

```
bubble_t teas.join('cafe', discounts, 'location')
```

Match rows in
this table...

...using values in
this column ...

...with rows in this
second table...

...using values in
this column.

join Example

bubble_t teas

cafe	drinks	prices
Gong Cha	Matcha Tea Latte	5.75
Tea Magic	Oolong Milk Tea	8
Hey Tea	Coconut Mango Boom	6.49
Moge Tee	Taro Milk Tea	7.45

discounts

% off	location
10	Gong Cha
25	Hey Tea
5	Moge Tee

```
bubble_t teas.join('cafe', discounts, 'location')
```

Match rows in this table...

...using values in this column ...

...with rows in this second table...

...using values in this column.

output:

cafe	drinks	prices	% off
Gong Cha	Matcha Tea Latte	5.75	10
Hey Tea	Coconut Mango Boom	6.49	25
Moge Tee	Taro Milk Tea	7.45	5

join Example

bubble_t teas

cafe	drinks	prices
Gong Cha	Matcha Tea Latte	5.75
Tea Magic	Oolong Milk Tea	8
Hey Tea	Coconut Mango Boom	6.49
Moge Tee	Taro Milk Tea	7.45

discounts

% off	location
10	Gong Cha
25	Hey Tea
5	Moge Tee

```
bubble_t teas.join('cafe', discounts, 'location')
```

Match rows in this table...

...using values in this column ...

...with rows in this second table...

...using values in this column.

output:

cafe	drinks	prices	% off
Gong Cha	Matcha Tea Latte	5.75	10
Hey Tea	Coconut Mango Boom	6.49	25
Moge Tee	Taro Milk Tea	7.45	5

Booleans and Comparisons

Boolean Data Type

- Booleans are data types for truth values: **True** or **False**
 - **True** is equivalent to `1`
 - **False** is equivalent to `0`
- `bool(x)` turns `x` into a boolean
 - e.g., `bool(1)` evaluates to **True** and `bool(0)` evaluates to **False**

Comparison Operators

Operation	Meaning
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to
==	equal to
!=	not equal to

Comparison Operators

Example	Result
$3 > 2$	
$3 > 3$	
$4 \leq 4$	
$2 < 4 < 7$	

Comparison Operators

Example	Result
3 > 2	True
3 > 3	False
4 <= 4	True
2 < 4 < 7	True

Comparison Operators

Example	Result
'4' == 4	
1 == 4 - 3	
2 != 2	
3 < 10 > 1	

Comparison Operators

Example	Result
<code>'4' == 4</code>	False
<code>1 == 4 - 3</code>	True
<code>2 != 2</code>	False
<code>3 < 10 > 1</code>	True

Comparisons with Arrays

- Single values can be compared against each element in an array
- Comparing two arrays will compare element-by-element

DEMO

and, or, and not

- You can combine conditional statements using **and** & **or**
 - **and** will return **True** if **all** expressions are **True** (and **False** otherwise)
 - **or** will return **True** if **any** expression is **True** (and **False** otherwise)
- You can negate a boolean value using **not**
 - **not True** will evaluate to **False**
 - **not False** will evaluate to **True**

and, or, and not

Example	Result
True and True	True
True and False	False
True or False	True
False or False	False
not False	True

Aggregating Comparisons

- Summing an array or list of bool values will count the **True** values only

Example	Result
True + False + True	2
1 + 0 + 1	2
sum ([True , False , True])	2

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	
<code>sum(z == 'Tim')</code>	
<code>np.arange(3) >= 1</code>	
<code>sum(np.arange(3) >= 1)</code>	
<code>sum(np.arange(4) <= 1) == 2</code>	

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	<code>array([True, False, True])</code>
<code>sum(z == 'Tim')</code>	
<code>np.arange(3) >= 1</code>	
<code>sum(np.arange(3) >= 1)</code>	
<code>sum(np.arange(4) <= 1) == 2</code>	

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	<code>array([True, False, True])</code>
<code>sum(z == 'Tim')</code>	2
<code>np.arange(3) >= 1</code>	
<code>sum(np.arange(3) >= 1)</code>	
<code>sum(np.arange(4) <= 1) == 2</code>	

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	<code>array([True, False, True])</code>
<code>sum(z == 'Tim')</code>	2
<code>np.arange(3) >= 1</code>	<code>array([False, True, True])</code>
<code>sum(np.arange(3) >= 1)</code>	
<code>sum(np.arange(4) <= 1) == 2</code>	

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	<code>array([True, False, True])</code>
<code>sum(z == 'Tim')</code>	2
<code>np.arange(3) >= 1</code>	<code>array([False, True, True])</code>
<code>sum(np.arange(3) >= 1)</code>	2
<code>sum(np.arange(4) <= 1) == 2</code>	

Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

Comparison Expression	Value
<code>z == 'Tim'</code>	<code>array([True, False, True])</code>
<code>sum(z == 'Tim')</code>	2
<code>np.arange(3) >= 1</code>	<code>array([False, True, True])</code>
<code>sum(np.arange(3) >= 1)</code>	2
<code>sum(np.arange(4) <= 1) == 2</code>	True

Control Statements

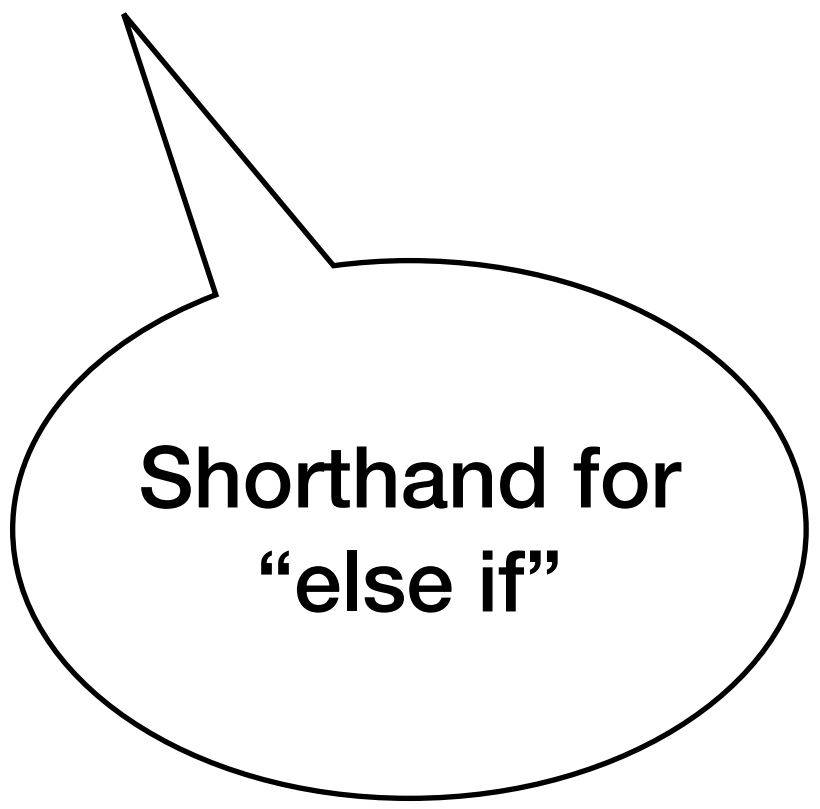
Control Statements

Control Statements modify *if* and/or *how many times* a block of code is executed in a program

- Two major types are **if** and **for**
 - **if** statements specify code that should be run conditioned on something being true
 - They can also specify if alternative code should be run otherwise
 - **for** loops allow executing code over each element in some sequence of items

if statements

- Conditionals begin with an **if** followed by a boolean statement
 - Runs code based on whether a boolean statement evaluates to **True**
- Conditionals can include a combination of **if**, **elif**, and **else** clauses
 - Maximum of one **if** and one **else**



Shorthand for
“else if”

if statements

```
if statement_1:
```

```
    first_code_block
```

if statements

```
if statement_1:
```

```
    first_code_block
```

```
elif statement_2:
```

```
    second_code_block
```

if statements

if statement_1:

first_code_block

else:

second_code_block

if statements

if statement_1:

first_code_block

elif statement_2:

second_code_block

else:

third_code_block

if statements

if statement_1:

first_code_block

elif statement_2:

second_code_block

elif statement_3:

third_code_block

else:

fourth_code_block

Demo: If Statements

Next Class

- Today
 - Joining Tables
 - Conditionals
- Wednesday
 - For Loops